

AI Engineer

Technical Exercise

Requirement

We maintain a catalogue of economic and industry time series. Each record defines one series — what it measures, for whom, at what frequency, in what unit. A chatbot, an API and a web interface all read from that catalogue, so it has to be correct, queryable, and able to absorb new subject areas without someone writing fresh application code for each one.

Attached is **series_catalogue_raw.csv** — 369 records, exported exactly as they sit in our hand-maintained source today. The fields are:

- **Identifier** — unique per record.
- **Title** — the human-readable name of the series. Also unique.
- **Category, SubCategory, Subset** — the browse classification.
- **Frequency, Unit, Currency** — how the series is published.
- **Discontinued** — whether the series is still being updated.
- **Parent, ChildTree1, ChildTree1_Name, ChildTree2, ChildTree2_Name** — the hierarchy. **Parent** holds the Identifier of the record above this one; the **ChildTree** columns hold comma-separated Identifiers of the records below it, and the matching **_Name** column labels that group of children.

The file covers two subject areas — civil aviation on-time performance, and merchandise trade quick estimates. They were authored at different times by different people, and are described differently as a result.

Problem statement

Build an end-to-end path: **ingest** → **storage** → **index** → **query** → **agent**. Use SQLite, Postgres, Mongo, Elasticsearch or anything else you prefer; we have no interest in your infrastructure choices beyond your reasons for them.

Given a user's question in plain English, return the catalogue records that answer it, ordered by relevance. Two entry points, both required:

- **python ingest.py --input series_catalogue_raw.csv** — loads the file into your store.
- **search(query, k=10, filters=None)** — returns ranked records. Expose it as a CLI or an HTTP endpoint, your choice. Each result should carry at least Identifier, Title, Category, SubCategory, Frequency and Unit.

Acceptance cases

Run these five. Put the **actual** output in your README — including where it is not what you hoped for. A case you got wrong and diagnosed is worth more to us than one you quietly dropped.

Query	What we are looking for
"IndiGo on-time performance at Delhi"	Exactly one record: IFCAOTPIND11M .
"monthly cashew exports in dollars"	EXMTXDCHWQ11M ranked first. Four cashew export records exist. Tell us what separates them and why yours wins.
"IndiGo punctuality"	All 12 IndiGo on-time records. State the order you return them in and defend it.
"how were flights in December?"	Your call entirely. Tell us what your system does, and why that is the right behaviour.
"Chennai cargo tonnage"	We hold no cargo tonnage data at all. Show us what comes back.

Guidelines

1. Ingestion and storage design. A normalised schema — decide what is a table or collection, what is denormalised for read speed, and what stays in the source of truth only. What did you turn into a structured field, and what stayed free text? Where did you draw that line, and why there? Include a cleaning layer, and state at minimum how you handle the same real-world thing named inconsistently, records flagged discontinued, and the tree columns. Finally: where does a new airline or a new commodity get added, and does that need a code change?

2. Index and query. What is your scoring function? How do you break ties between records that differ in only one attribute? What is your index refresh strategy, and what does a query see during a rebuild?

3. Vector or hybrid retrieval, if you use it. Which embedding model, its dimensionality, and why that one. What text do you embed — Title alone, or Title plus the categorical fields? One vector per record, per field, or per chunk? Justify it; this choice usually matters more than the index type. If you go hybrid, evaluate it against keyword-only and vector-only rather than assuming it wins.

4. The agent layer. A thin LLM layer over **search()**. It should resolve the question to records in your store, never answer from its own knowledge. What does it do when nothing matches? What does it do when the question is ambiguous?

5. Good to have, entirely optional. Filtered search — **search(q, filters={"Frequency": "Monthly"})**. Did-you-mean over the catalogue vocabulary. How you would prove hybrid beats lexical in production, and what your guardrail metric would be.

What to send us

A git repository (or a zip) containing your code, and a **README** of roughly two to four pages. We will read the README before we read the code. Alongside the five acceptance cases, cover:

- One thing you got wrong first and then changed. What made you change it?
- What you did not do, and what you would do next if you had another day.

One extension question

Answer this in the README from your design alone. **Do not build it.**

Next quarter we expect to add airport cargo tonnage: monthly figures, by airport and by cargo type (international, domestic, total), from the same source. What changes? Be specific about your schema, your ingestion step and your index — and tell us what you would have done differently in the first place had you known.

On AI tools

Use them. We do, and we would think less of an engineer who didn't. Nothing here needs to be hand-written, and there is no penalty of any kind for AI-assisted work.

What we are assessing is your judgement — which decisions you made, which you rejected, and why. Expect to defend every choice in the follow-up conversation, including the ones you delegated to a model. That is the only thing being measured.

Timing

Please send this back **within three days** of receiving it. That window exists so you can fit the work around a job, not as a signal of how much work we expect — the exercise is deliberately small. We would far rather see a narrower submission with an honest account of what you left out than a complete one you cannot explain, so if you catch yourself polishing, stop and write instead.

How we assess it

Modelling and retrieval decisions, and the reasoning behind them	50%
The acceptance cases, and how honestly you report them	20%
The extension question	20%
Code quality	10%

Note the weighting. A rough submission that is well argued scores higher here than a polished one that arrives without its reasoning.

The follow-up conversation

We will book 30–45 minutes to go through your submission with you. Expect us to push back on decisions — including ones you got right. We are interested in how you hold or revise a position under challenge, not in whether you end up agreeing with us.

Submission

Reply to us with a link to your repository, or a zip of it.

Where the data is ambiguous — and it is, in places — make a call and write down what you assumed. Recording the assumption is part of what we are asking for.